课程笔记

无锁编程：细粒度锁、比较并交换与 ABA 问题

CMU 15-418/618 并行计算机架构与编程 · Lecture 21（Spring 2018）

lecture-to-notes

2026 年 10 月 28 日



视频作者/频道：CMU 15-418/618（B 站搬运：alexphil）
发布日期：2018 年春季学期
视频时长：1 小时 21 分 51 秒
视频链接：`https://www.bilibili.com/video/BV17N411B71Z/`

目录

1 为什么不用一把大锁 **2**

1.1 动机：互斥量太重 2

1.2 自旋锁的适用范围 2

1.3 三个层次 3

1.4 本章小结 3

2 比较并交换：无锁的基石 **3**

2.1 原子操作家族 3

2.2 三参数语义 4

2.3 循环重试与无限循环 5

2.4 用 CAS 实现锁 5

2.5 本章小结 5

3 语言封装：C11 的原子类型 **6**

3.1 本章小结 7

4 并发链表的三种方案 **7**

4.1 串行版本 7

4.2 并发时出了什么问题 8

4.3 方案一：一把锁 8

4.4 本章小结 9

5 方案二：手递手细粒度锁 **9**

5.1 核心不变式 9

5.2 顶层锁的作用 10

5.3 实用变体：哈希桶 11

5.4 代价 11

5.5 本章小结 12

6 无锁算法：定义与两个实例 **12**

6.1 基于锁方案的根本弱点 12

6.2 单读单写有界队列 13

6.3 单读单写无界队列 14

6.4 本章小结 16

7 ABA 问题 **16**

7.1 看起来应该可行的做法 16

7.2 问题的形态 17

7.3 应对方向 17

7.4 本章小结 18

8 总结与延伸 **18**

1 为什么不用一把大锁

讲者开场就给了个提醒：这是课程中材料相当棘手的部分。你看着这些代码会隐约觉得懂了，但最好回去再核对一遍。

1.1 动机：互斥量太重

问题的起点很实际：pthread 互斥量是重量级代码，它需要与操作系统交互，可能让作业被挂起、放入队列、之后重新唤醒。

这个过程可能耗时 20000 个时钟周期以上。因此你不希望在并发程序的应用级数据结构上做这件事。

锁的开销量级决定了它的适用范围

第 20 讲区分过：纯硬件原子操作在几十到几百周期，涉及操作系统调度的在数千周期以上。而 pthread 互斥量作为最重的同步原语，可以到数万周期。用它保护一个每次只做几条指令的临界区，开销会比临界区本身大几个数量级。

1.2 自旋锁的适用范围

有时自旋锁是合适的：如果你只想抓住某个东西足够久，完成几次更新而不让别人来搅局，自旋锁是很有效的方式。

但无论用哪种锁，若把锁加在整个数据结构上——比如一个哈希表——你就把对该表的所有访问串行化了，这会使它不适合并行编程。

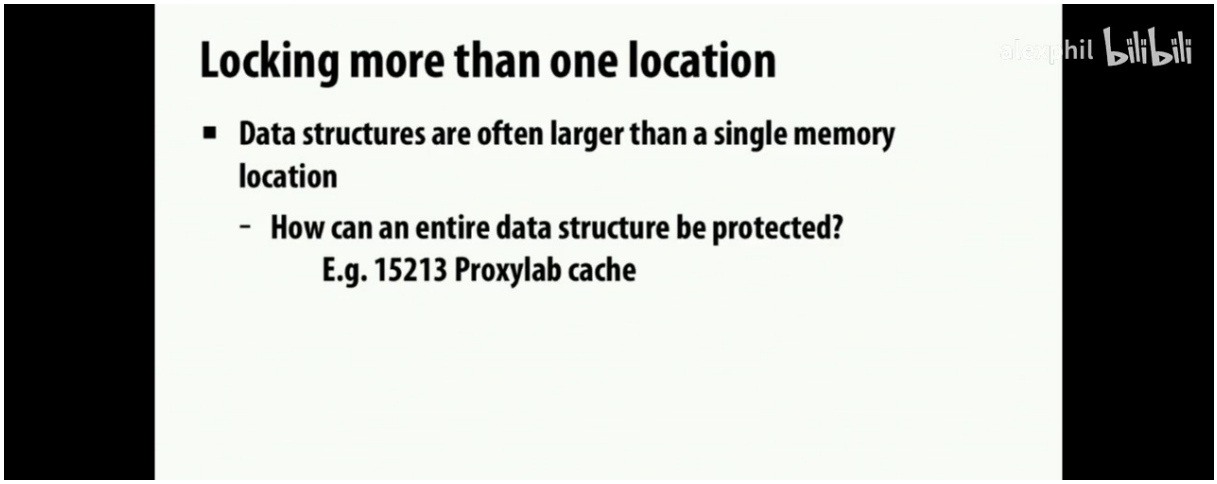


图 1：锁定多个位置：数据结构往往大于一个内存位置[1]

[1] 视频画面时间区间：00:17:00–00:17:15。若为裁剪图，时间区间仍对应原视频画面。

1.3 三个层次

三层方案对应三种不同的失败模式

一把大锁的失败模式是串行化——正确但无并行。细粒度锁的失败模式是死锁与实现错误——并行但容易出错。无锁的失败模式是 ABA 与回收难题——无阻塞但极难正确实现。选择层次时，要先明确自己更怕哪种失败。

讲者把改进方向分成三层。最粗的是用一把大锁；再细一步是给数据结构的各个组成部分分别加锁；最极端的是完全去掉锁，另找机制保证共享与更新的正确性。

最后这一层就是无锁编程，这个领域有相当多的研究成果。

1.4 本章小结

pthread 互斥量开销可达数万周期，不适合应用级数据结构。自旋锁适合短暂持有，但把锁加在整个结构上会串行化所有访问。改进方向有三层：一把大锁、细粒度锁、完全无锁。

下一章从最基础的原语开始。

2 比较并交换：无锁的基石

2.1 原子操作家族

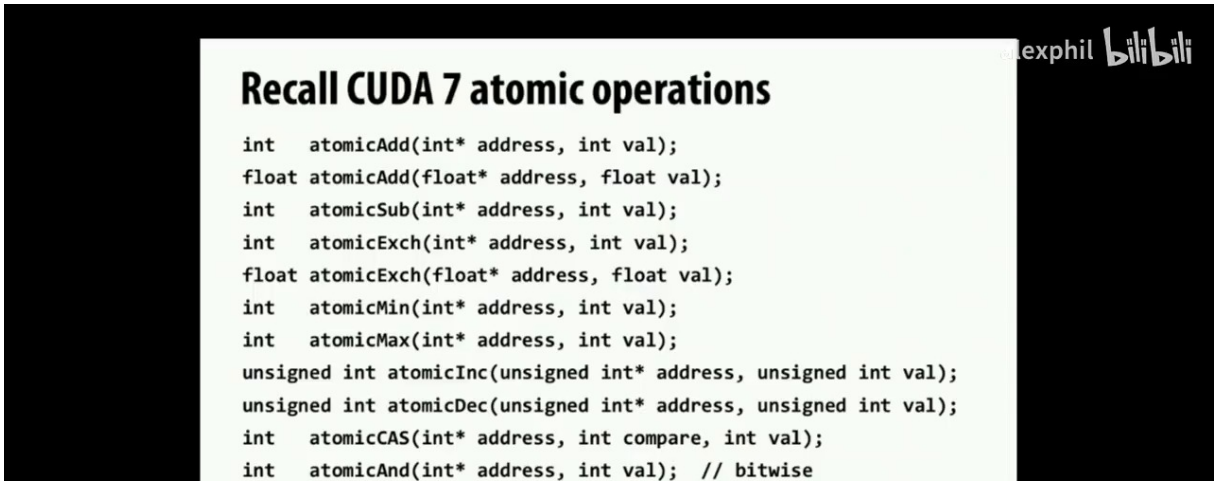


图 2: 回顾 CUDA 的原子操作：原子加、原子自增等[2]

细粒度方案依赖原子操作。讲者展示了 CUDA 提供的一组原子操作（GCC 也有类似的列表），包括各种原子加、原子自增或执行某种简单算术运算的方式。

[2]视频画面时间区间：00:05:00–00:05:15。若为裁剪图，时间区间仍对应原视频画面。

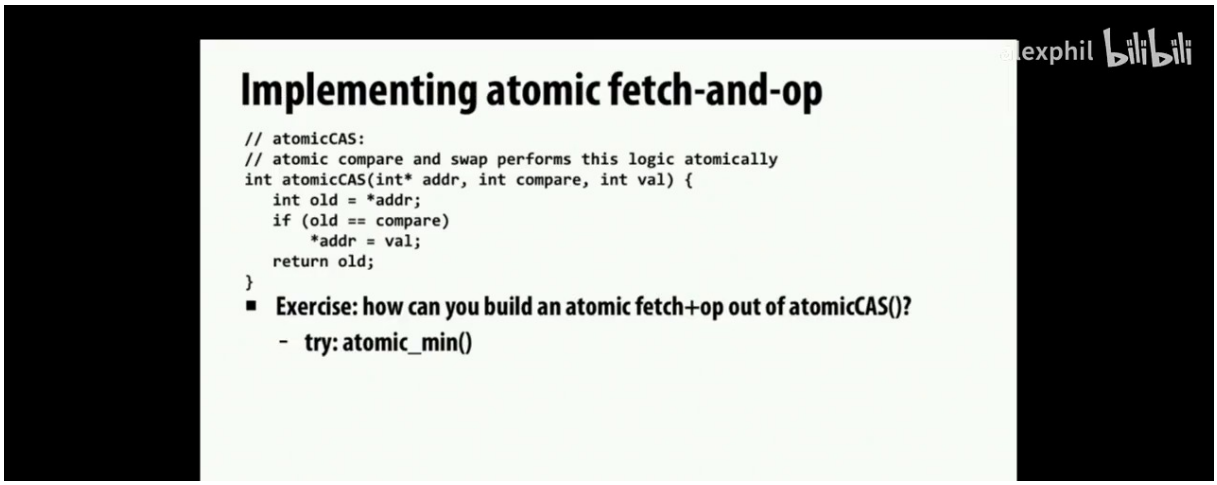


图 3: 实现原子取并运算：原子 CAS 的接口[3]

另一类叫比较并交换，讲者说这些实际上是硬件实现的核心基础。

2.2 三参数语义

比较并交换是一个三参数操作：第一个是地址，第二个是你认为该地址上当前是什么值，第三个是你想存进去的值。

整个操作原子地完成，也就是说这些步骤被归在一起执行。具体是：先检查该地址当前的值为多少，称为 old；若它与期望值匹配，就执行更新，并把 old 作为结果返回。

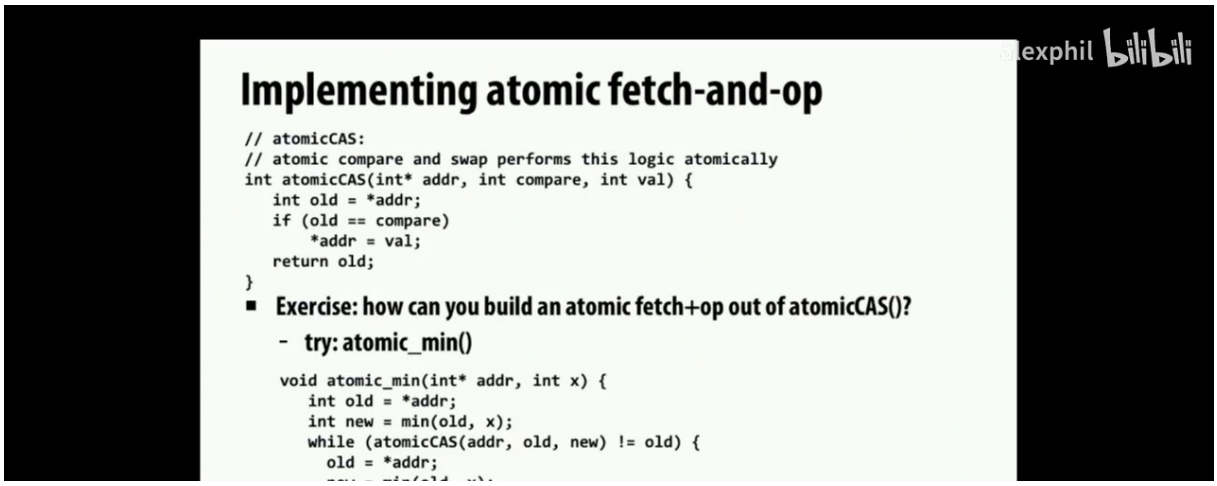


图 4: 用 CAS 循环实现原子更新：反复尝试直到成功[4]

讲者补充说，设计上你会确保要写入的新值与期望值不同，否则这个操作没什么意义。而当 old 返回后若与你传入的比较值不匹配，你就知道这次操作失败了。

用公式表达即：

$$\mathrm{CAS}(addr, expected, new) = \begin{cases} \text{成功，写入 } new, \&\text{若 } \mathrm{mem}[addr] = expected \\ \text{失败，不写入}, \&\text{否则} \end{cases}$$

[3] 视频画面时间区间：00:05:30–00:05:45。若为裁剪图，时间区间仍对应原视频画面。

[4] 视频画面时间区间：00:09:40–00:09:55。若为裁剪图，时间区间仍对应原视频画面。

- 返回值总是该地址原来的内容，因此可用它判断成败；
- 成功条件依赖” 当前值恰好等于期望值”，这正是 ABA 问题的根源；
- 操作整体不可分割，期间没有其他线程能观察到中间状态。

2.3 循环重试与无限循环

CAS 通常在循环里使用：不断尝试直到成功。
讲者提醒说，最好把这段代码改一下使它不会无限循环——这通常是个好习惯。
他还说，实现原子自增会更合理一些，因为那样就没有无限循环的可能。

CAS 重试循环可能不终止

在极高争用下，一个线程可能反复失败。若不加超时或退避，理论上可能长期无法推进。虽然实际中概率很低，但在需要确定性响应的系统（实时系统、持有锁期间被换出）里，这种无界重试是不可接受的。

2.4 用 CAS 实现锁

讲者指出，用这个原语实现锁也很直接：假设某地址上有一个标志，你不断尝试把一或零换进去，并检查旧值是否等于一。

锁与原子操作是同一种原语的两种用法

CAS 既能实现” 读-改-写” 这类无锁更新，也能实现互斥锁。区别在于用途：前者让多个线程都尝试推进数据结构，后者只允许一个线程进入临界区。因此无锁编程并不是抛弃原子原语，而是改变使用它的方式。

2.5 本章小结

比较并交换是三参数原子操作：地址、期望值、新值，返回旧值以判断成败。它通常在循环中重试，需注意无限循环风险。用它可以实现原子自增与自旋锁。它是硬件实现所有同步的核心基础。

有了原语，先看语言层面的封装。

3 语言封装：C11 的原子类型

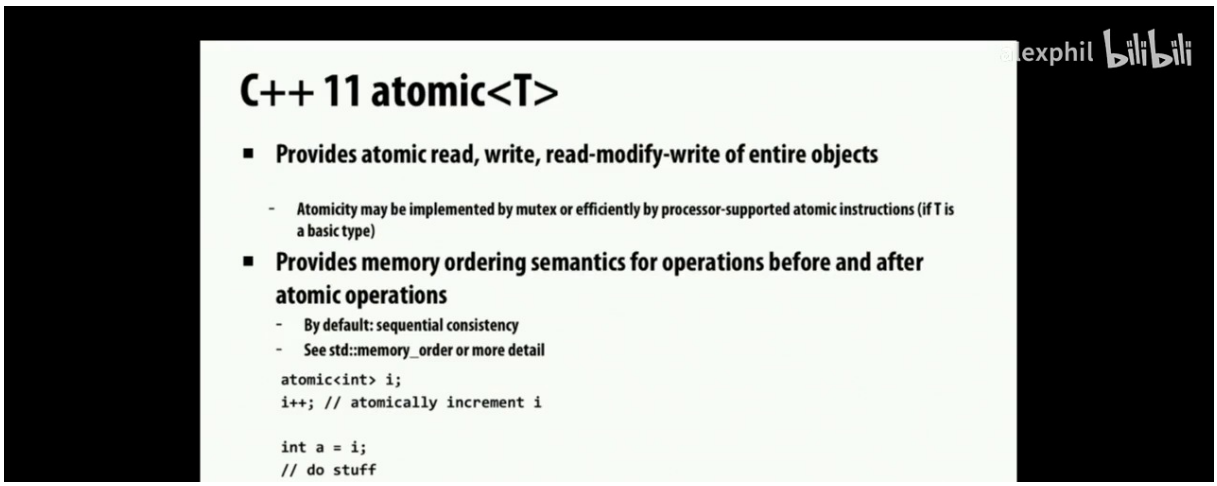


图 5: C++11 的 atomic 模板：可把任意类型做成原子[5]

讲者指出 C++ 的 C11 版本有一个有意思的能力：通过模板类型把任何东西做成原子的。

对 int 或任何基本数据类型使用时，实现会利用为支持原子操作而设计的原生指令。比如对原子变量做自增，通常会用到取并加之类的操作。

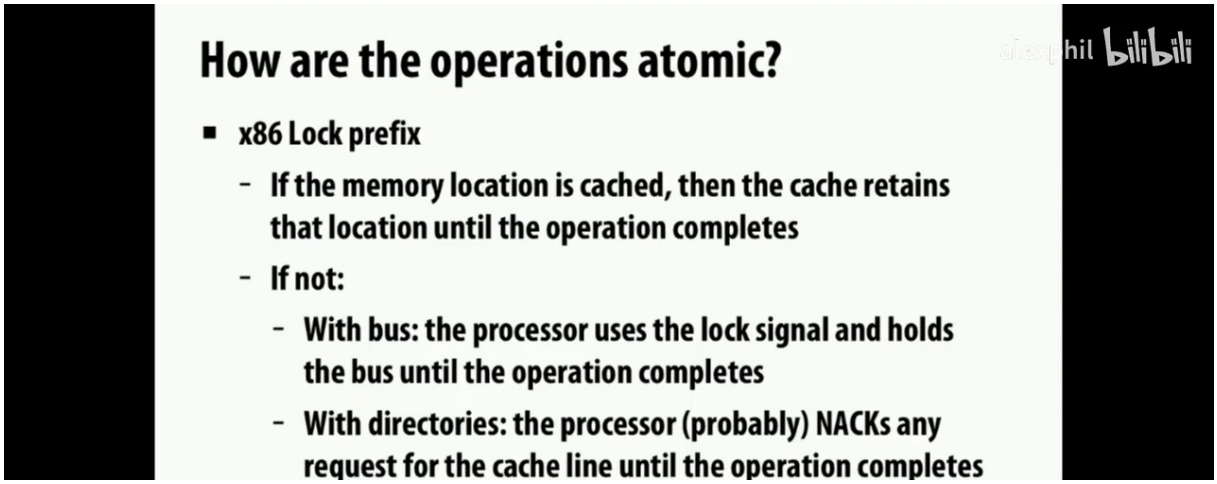


图 6: 这些操作如何做到原子：x86 的 lock 前缀[6]

对于更复杂的类型，你可以对几乎任何数据类型这么做；但若是庞大复杂的数据结构，它就干脆在里面放一把大锁。讲者说它会尽量聪明地选择内部实现，尤其是对标准数据类型。

抽象层的实现可能是”聪明”也可能是”退化为锁”

同一个 atomic 接口，对基本类型映射到单条原子指令，对复杂结构则可能退化为内部加锁。因此不能假设”用了原子类型就一定无锁”。要判断实际行为，需要看类型大小与平台实现，必要时用工具验证。

[5] 视频画面时间区间：00:11:00–00:11:15。若为裁剪图，时间区间仍对应原视频画面。

[6] 视频画面时间区间：00:12:30–00:12:45。若为裁剪图，时间区间仍对应原视频画面。

3.1 本章小结

C11 的 atomic 模板把原子性抽象成类型，对基本类型映射到原生指令，对复杂结构则可能退化为加锁。这个抽象很方便，但不能假设它总是无锁实现。

下面进入本讲的主体：并发数据结构。

4 并发链表的三种方案

4.1 串行版本

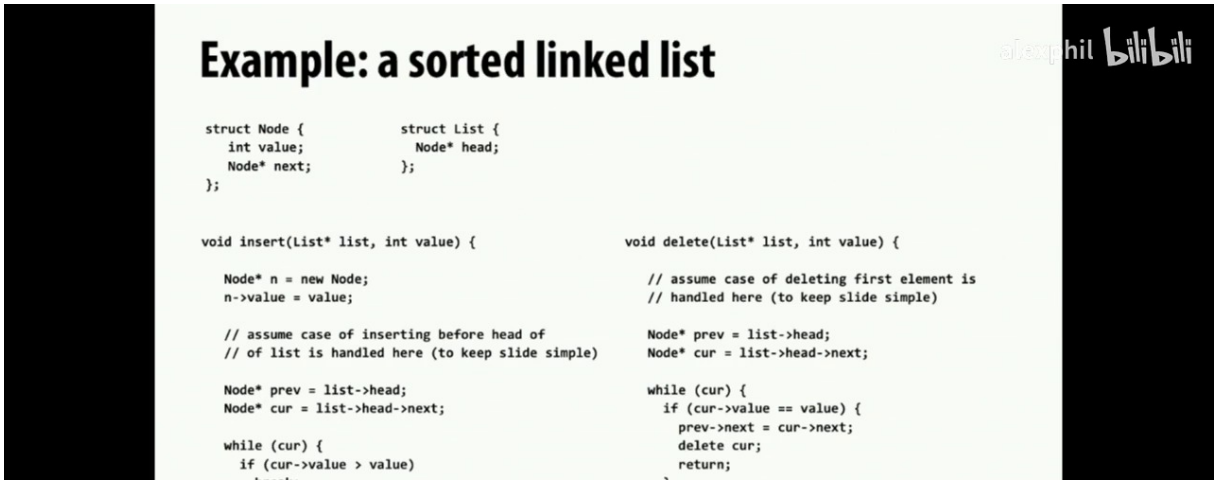


图 7: 示例：有序链表的节点结构与列表结构[7]

讲者用标准有序链表代码作为例子。插入时沿链表走，直到遇到大于待插入值的位置；此时已创建好新节点 n，把它插入：cur 成为 n 的后继，而 cur 原来的前驱 prev 改为指向 n。

删除同理：沿链表走到匹配值处，摘除并释放该元素。

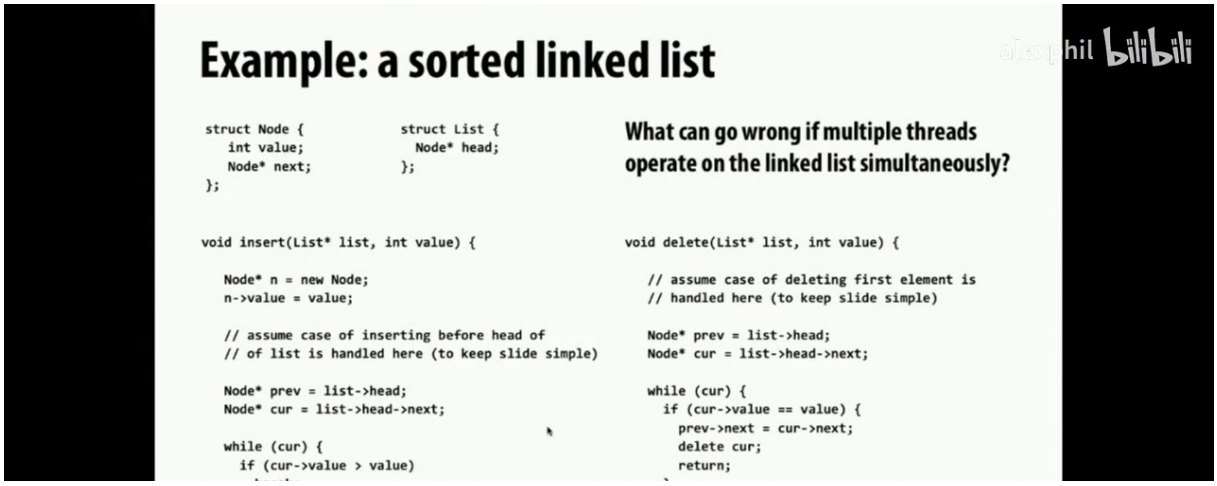


图 8: 有序链表的插入：把新节点接到前驱与后继之间[8]

[7]视频画面时间区间：00:18:00–00:18:15。若为裁剪图，时间区间仍对应原视频画面。

[8]视频画面时间区间：00:19:30–00:19:45。若为裁剪图，时间区间仍对应原视频画面。

4.2 并发时出了什么问题

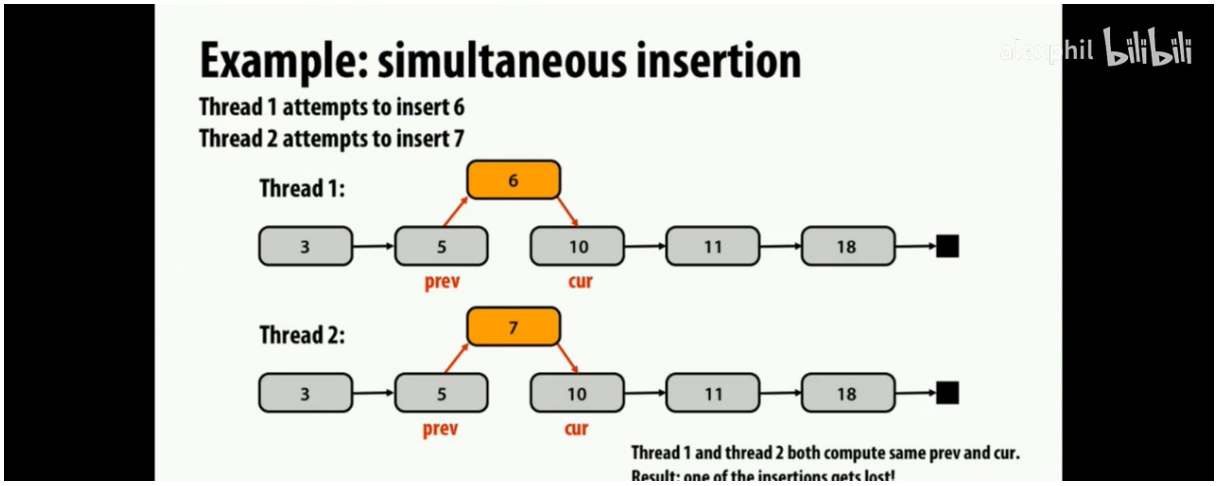


图 9: 同时插入：线程 1 插入 6，线程 2 插入 7[9]

讲者给出具体场景：一个线程想插入元素 6，另一个想插入 7。

线程 1 先到达并开始拼接；与此同时线程 2 到达同一位置也开始拼接。于是两者竞争谁真正把元素接进链表。

例子里显示一个插入成功、另一个没有。讲者说其实基本只有这两种结果，因为这段代码中唯一被更新的共享部分是那个前驱指针。

删除也有类似问题：一个线程已经准备好删除，同时有插入发生；或者两个删除作用于相邻元素。各种问题都会出现。

4.3 方案一：一把锁

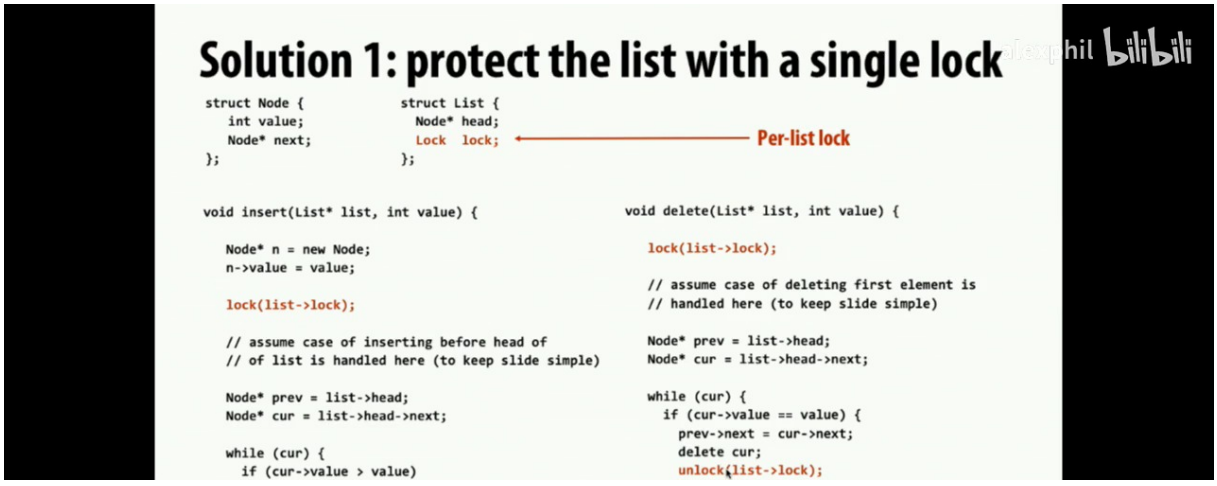


图 10: 方案一：用一把锁保护整个链表[10]

最快捷的做法是给整个结构加一把锁：只要有一方在遍历这个链表，它就持有整条链表的锁，直到完成操作才释放。

[9] 视频画面时间区间：00:20:30–00:20:45。若为裁剪图，时间区间仍对应原视频画面。

[10] 视频画面时间区间：00:22:00–00:22:15。若为裁剪图，时间区间仍对应原视频画面。

讲者提醒一个细节：代码里若有多个返回点，必须确保每条路径都解锁，不能有些解有些不解。但坏处正如前面所说：这让链表变成完全串行的数据结构。

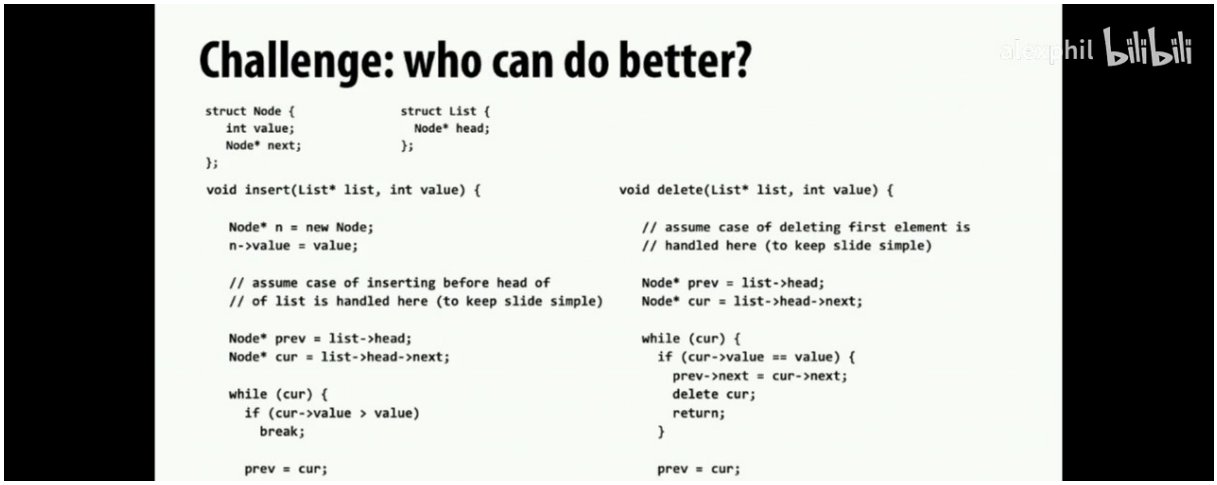


图 11: 挑战：谁能做得更好[11]

4.4 本章小结

有序链表的并发问题集中在拼接时的前驱指针更新，两个线程会竞争同一个位置。最简方案是用一把锁保护整个链表，正确但完全串行，且需注意多返回点的解锁。

这引出”谁能做得更好”的挑战。

5 方案二：手递手细粒度锁

5.1 核心不变式

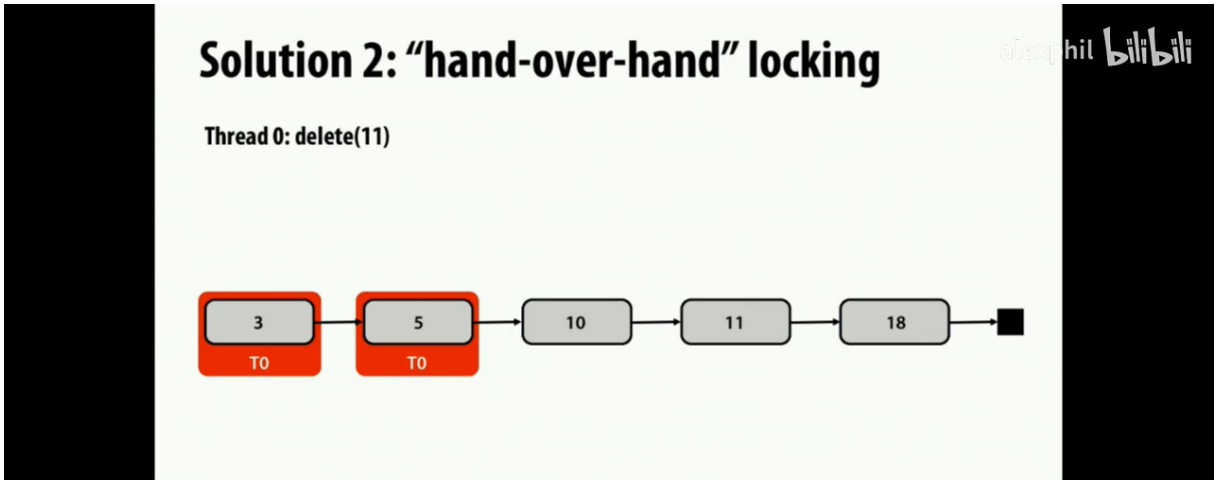


图 12: 方案二：手递手加锁遍历，同时持有前驱与当前两把锁[12]

[11] 视频画面时间区间：00:22:30–00:22:45。若为裁剪图，时间区间仍对应原视频画面。

[12] 视频画面时间区间：00:23:00–00:23:15。若为裁剪图，时间区间仍对应原视频画面。

讲者展示手递手遍历的过程：遍历循环有一个循环不变式——循环开始时保证持有前驱与当前这两把锁。

随着循环推进，它释放一把再获取下一把。因此会有一个瞬间只持有一把锁，但 ** 永远不会有不持锁的瞬间 **。



图 13: 方案二：细粒度加锁，每个节点一把锁[13]

手递手的正确性来自”始终持有至少一把锁”

若允许出现不持锁的瞬间，另一个线程就可能在你松手的位置插入或删除节点，使你的前驱指针失效。始终保持至少一把锁，保证了你脚下的路径不会被抽走。这个不变式比”每步都加锁”更强，也更精确。

5.2 顶层锁的作用

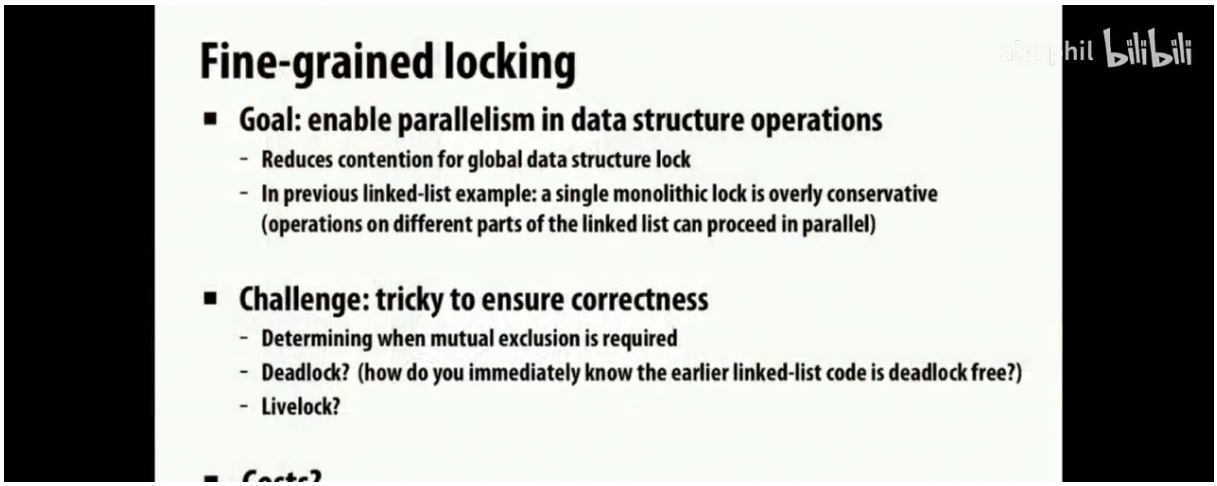


图 14: 细粒度加锁的目标：让数据结构操作的并行化成为可能[14]

[13] 视频画面时间区间：00:27:00–00:27:15。若为裁剪图，时间区间仍对应原视频画面。

[14] 视频画面时间区间：00:31:30–00:31:45。若为裁剪图，时间区间仍对应原视频画面。

讲者解释为什么还需要一把覆盖整个结构的锁：它是用来控制初始访问的，让所有人一开始被串行化，进入顺序决定了它们沿链表推进的次序。

但这把锁只持有到你取得第一个元素的锁为止，然后释放——这就是手递手。别人可以跟在你后面进来，但不能越过你。



图 15: 练习：实现一个细粒度锁定的二叉搜索树[15]

5.3 实用变体：哈希桶

讲者提到一个很实际的做法：用哈希表，每个桶是一条链表。这是实现哈希表的标准方式。

第一层哈希把争用分散开，使这些链表通常最多 10 到 20 个元素长。这样就解除了串行化，而且完全合理。

5.4 代价

细粒度锁的出错点随锁数量增长

讲者明确说，与一把大锁相比，细粒度方案有很多地方可以引入 bug，而且都是微妙的问题。锁越多，需要维护的不变式越多：谁持有哪把、何时释放、哪些路径可能提前返回。正确性论证的复杂度显著上升。

讲者还说明细粒度方案若用标准锁实现，开销高到不实用；而自旋锁通常只是一个整数，因此存储开销不算大问题。

并发度与锁的数量成正比，但受链表中元素数限制：

$$P_{\text{concurrent}} \approx \min\left(N_{\text{locks}}, N_{\text{threads}}\right)$$

- N_{locks}：数据结构上可独立加锁的位置数；
- N_{threads}：活跃线程数；
- 若链表很长而锁只在每个节点上，则可并发推进的线程数受链表长度限制；

[15] 视频画面时间区间：00:33:00–00:33:15。若为裁剪图，时间区间仍对应原视频画面。

- 哈希桶方案通过增加独立桶数来提高这个上界。

手递手在需要修改多个指针的结构上难度陡增

链表插入只改两个指针，删除只改一个。而二叉搜索树的删除可能涉及节点替换、子树搬移，需要同时持有多个节点的锁并按固定顺序获取以避免死锁。锁数量增加使”始终持有至少一把”这个不变式难以维护。

他也提到去年曾实现过一个完整的二叉搜索树手递手遍历版本，说它极其优雅但并不简单——尤其是删除本身已经棘手，再叠加手递手遍历。

5.5 本章小结

手递手遍历的不变式是始终持有前一与当前两把锁，因此路径不会被抽走。顶层锁只用于让进入者有序，取得第一个元素的锁后即释放。实用变体是用哈希桶把争用分散到短链表上。代价是出错点随锁数量增长，实现复杂度显著上升。

这些都是基于锁的方案。下一章看完全不用锁的思路。

6 无锁算法：定义与两个实例

6.1 基于锁方案的根本弱点

讲者指出基于锁的系统有一个可能被长期延迟的风险：某个进程被页换出、甚至崩溃。

若它崩溃，可能造成大麻烦；若被换出，整个访问会挂起等待该进程回来。这些都是操作系统层面的问题。

于是无锁算法的定义是：不会发生这种情况——至少有一个线程总能推进。

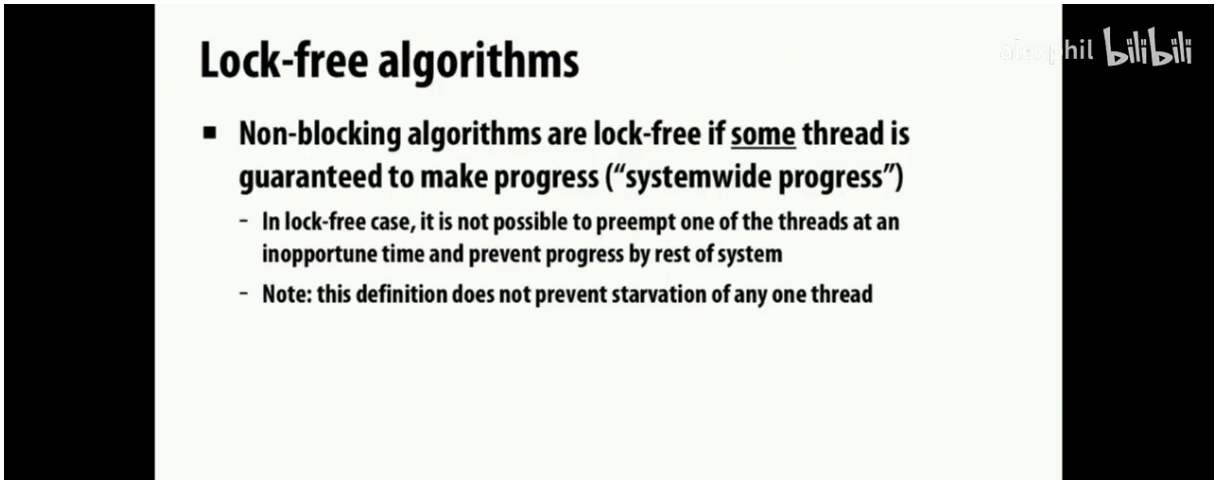


图 16: 无锁算法：若某些线程总能推进则为无锁[16]

讲者自己也承认这个保证有点宽松。因为用 CAS 循环时我们只是不断尝试直到成功，而在严重争用下可能所有人反复失败。

可以把它写成失败概率的形式：

[16] 视频画面时间区间：00:36:00–00:36:15。若为裁剪图，时间区间仍对应原视频画面。

$$P(\text{某线程成功}) \geq 1 - (1-p)^{N_{\text{retries}}}$$

- p：单次 CAS 尝试成功的概率，随争用加剧而下降；
- N_{retries}：某线程的重试次数；
- 只要 $p > 0$，重试次数增加时成功率趋于 1，但单次延迟无上界。

因此无锁给的是概率性进展，而不是时间上界。因此说” 绝对保证推进” 是过强的说法，实际含义是：以极高概率，某个线程能继续前进，且不受操作系统上下文切换问题的影响。

无锁的保证是” 系统层面” 而非” 个体层面”

无锁只保证至少有一个线程能推进，不保证某个特定线程会推进。这意味着单个线程可能饥饿，而整体系统仍在前进。若应用要求每个线程都有响应时间上界，无锁并不足够，还需要额外的公平性机制。

6.2 单读单写有界队列

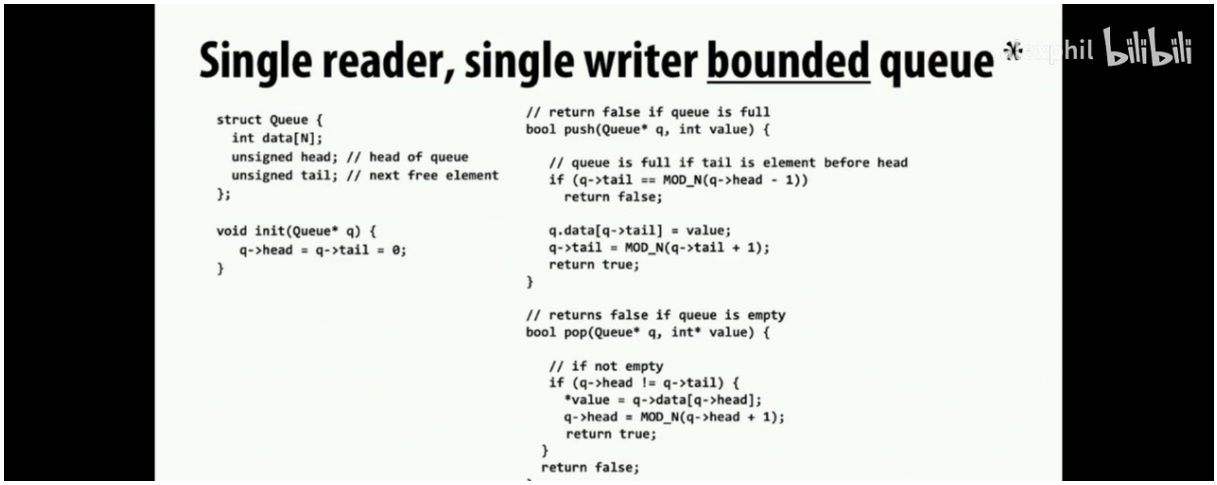


图 17: 单读单写的有界队列：环形缓冲区实现[17]

讲者给出一个很简单的例子，并声称这段代码可用——前提是只有一个读者和一个写者，且它们不是同一个线程：一个线程插入，一个线程删除。

结构是一个环形队列：head 表示最旧的值，tail 指向下一个应该插入的位置。插入就是写入该位置再把 tail 前移一格；弹出就是读出该位置的值再把 head 前移一格。两者都对 n 取模以回绕。

[17] 视频画面时间区间：00:37:00–00:37:15。若为裁剪图，时间区间仍对应原视频画面。

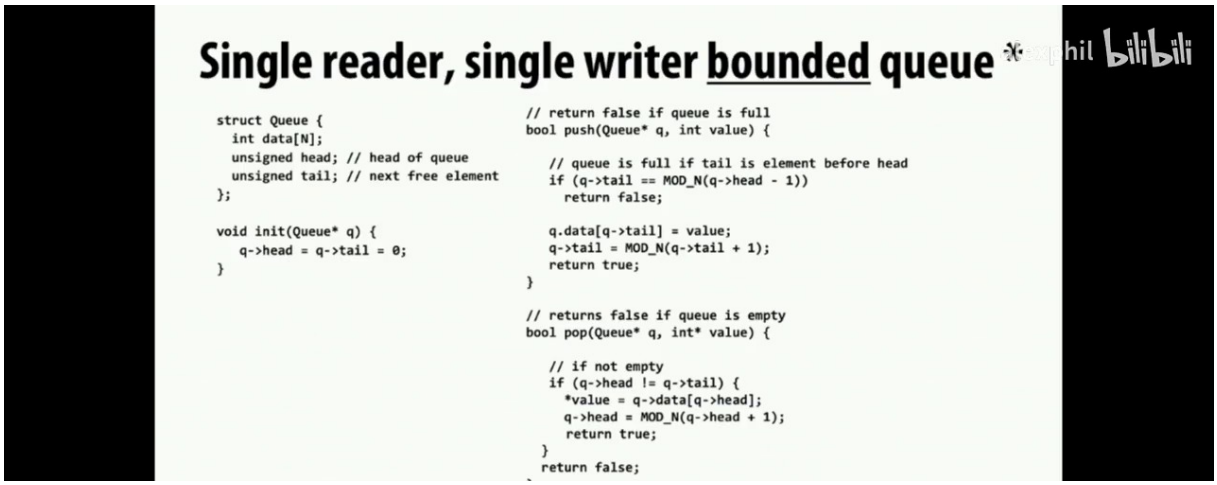


图 18: 队列满与空的判定：牺牲一个空位以区分两者[18]

这里有个经典问题：如何区分满与空？这段代码的做法是让出一个空位——永远不让 tail 越过 head。当 tail 到达 head 的位置时就宣布队列满，因此实际容量只有 n 减 1。

$$\text{capacity} = n - 1$$

- n：环形缓冲区的槽位数；
- 牺牲一格用于区分” 满” 与” 空” 两种状态；
- 否则 head 与 tail 相等时无法判断是哪一种。

讲者说明也可以用别的方案，但那样就需要另一个标志来告知空或满。

他最后指出这个方案的前提：对内存序做合理假设，且更新一个字时不会出现” 改了半个字” 导致读到不一致的值。它甚至不需要取模操作本身是原子的。

6.3 单读单写无界队列

[18] 视频画面时间区间：00:40:00–00:40:15。若为裁剪图，时间区间仍对应原视频画面。

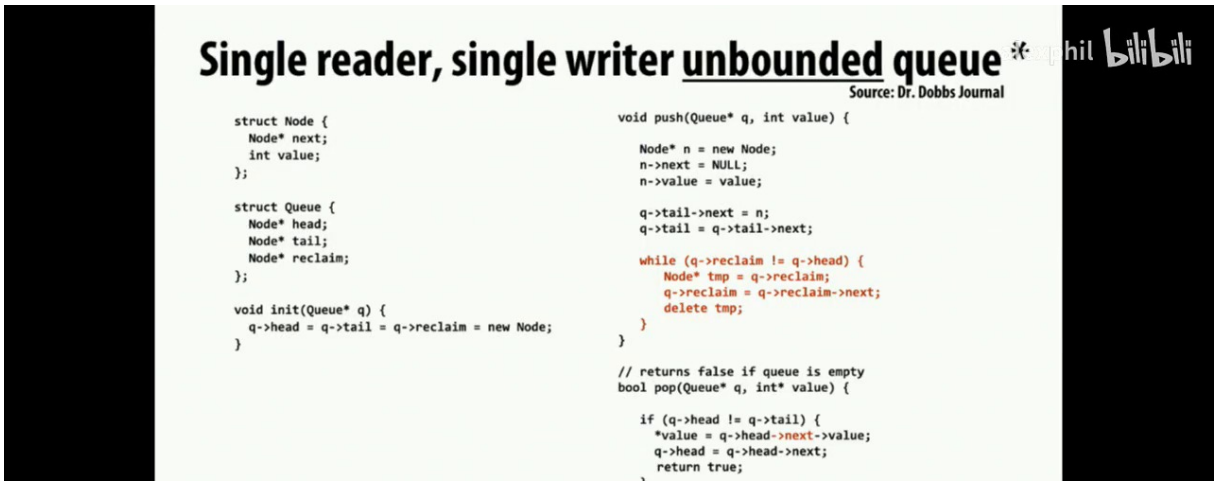


图 19: 基于链表的无锁队列：head、tail 与 reclaim 三个指针[19]

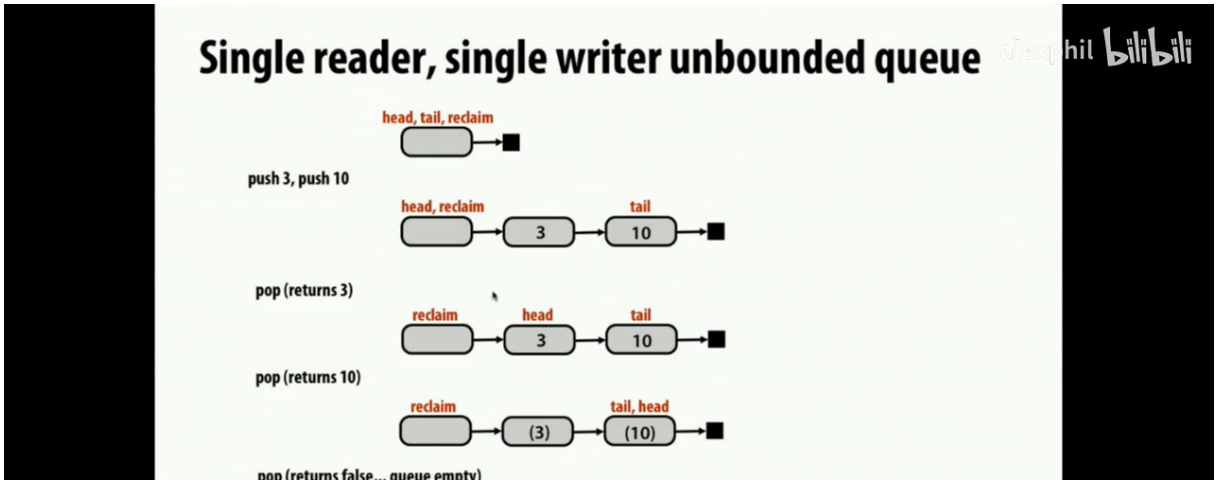


图 20: reclaim 指针：pop 只移动 head，从不真正删除节点[20]

讲者接着给出一个同样基于单读单写、但没有大小限制的方案，因为它用链表实现。

这个想法很有意思：队列有三个指针——标准的 head 与 tail，外加第三个叫 reclaim。

关键在于：pop 虽然从链表中移除元素，但 ** 从不真正删除任何节点 **。只有 pusher 才被允许用 new 创建节点、用 delete 释放节点。

head 是一个哑元素，从不会有值放进去。初始时是空链表，三个指针都指向同一个节点。

push 时不断往 tail 加节点。pop 只是让 head 沿链表前进一格，而不删除前一个元素。

reclaim 指针负责跟踪链表中还有哪些节点——即使它们已经不再是有效数据。

讲者用括号表示那些已被移除、但物理上仍留在链表元素中的值：它们只是残留值。

单写者负责回收是无锁队列的关键分工

若 pop 也能释放节点，那么一个慢的读者可能仍持有指向该节点的指针，而节点已被释放并被复用，于是读到垃圾数据——这正是 ABA 问题的温床。把分配与释放权限集中到唯一的写者，就消除了这类竞态。

[19] 视频画面时间区间：00:44:00–00:44:15。若为裁剪图，时间区间仍对应原视频画面。

[20] 视频画面时间区间：00:46:00–00:46:15。若为裁剪图，时间区间仍对应原视频画面。

6.4 本章小结

无锁算法保证至少有一个线程总能推进，且不受操作系统上下文切换影响；这个保证是系统层面而非个体层面。两个实例是单读单写的环形队列（牺牲一格区分满空）与单读单写的链表队列（只有写者可分配释放节点，pop 只移动 head）。

这些方案都限定了单读单写。放宽这个限制会引出更棘手的问题。

7 ABA 问题

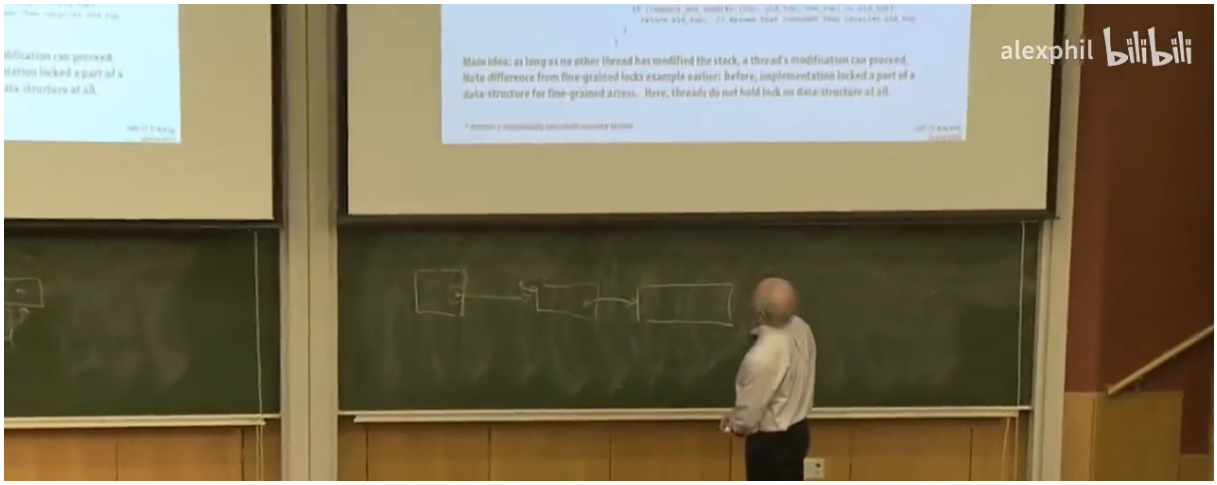


图 21: 从细粒度锁转向无锁实现时遇到的新问题[21]

7.1 看起来应该可行的做法

讲者描述了一个自然想法：我取得指向某个节点的指针，把它作为新栈顶，想把它塞进链表前端。但要确保在这个过程中没有别人插入新元素或删除那个元素。

讲者问：这看起来应该可行吧？然后自己回答：你大概猜到了，不行。

[21] 视频画面时间区间：00:54:30–00:54:45。若为裁剪图，时间区间仍对应原视频画面。

7.2 问题的形态

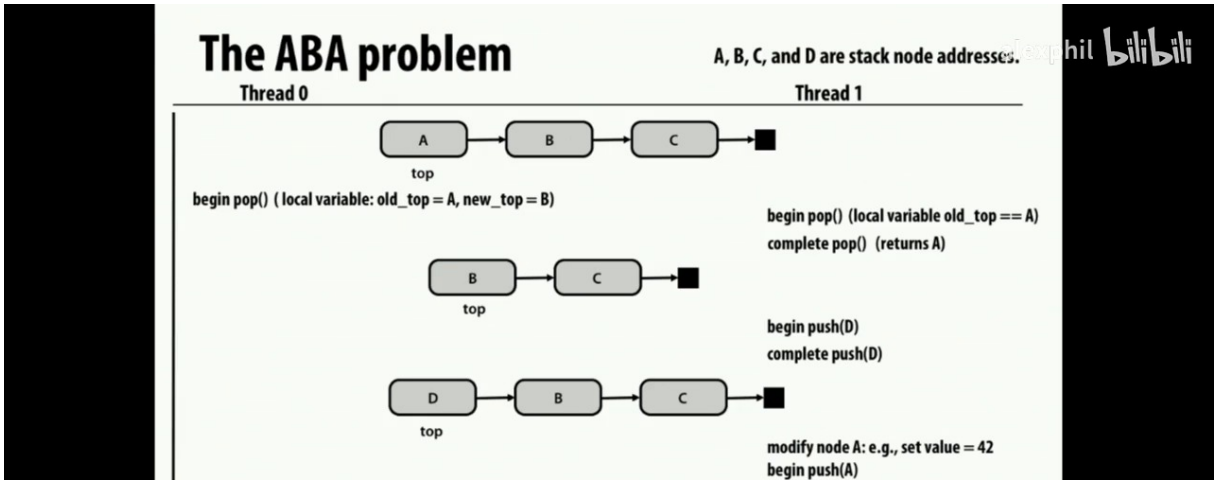


图 22: ABA 的具体场景：pop 后重新插入导致 CAS 误判[22]

这个问题的名字很简短：ABA 问题。做这行的人一听说” 我有个 ABA 问题”，立刻就知道你在说什么。

场景是这样的：两个线程都在操作这个栈。线程 0 开始一次弹出操作，它认为旧栈顶是 A、新栈顶是 B，想在做完比较并交换后确认自己是对的。

与此同时线程 1 完成了一次弹出并成功返回，真的从链表中移除了一个元素。然后线程 1 又插入了一些东西。

如果线程 1 插入的恰好是 A，那么当线程 0 执行 CAS 时，它看到栈顶仍是 A，就以为没有任何变化，于是成功地把栈顶设为 B。但此时 B 可能已经被释放了。

CAS 只能比较值，无法感知中间状态

ABA 的根源在于比较并交换只看” 当前值是否等于期望值”，而不看” 这个值是否一直是这个值”。若中间经历了 A→B→A 的变化，CAS 无法察觉，于是基于旧假设的更新被错误地接受。这是无锁算法与基于锁算法最本质的差别之一。

7.3 应对方向

讲者给出两类常见对策。其一是给指针附加一个版本计数，使每次修改都改变计数，因此 A→B→A 会变成 (A,1)→(B,2)→(A,3)，CAS 便能识别出变化。其二是把节点回收推迟到确定无人引用之后，例如让唯一的写者负责释放，或使用引用计数、风险指针等机制。

内存回收是无锁设计的核心难点

在有锁的世界里，” 删除一个节点” 只需在临界区内摘链并释放。在无锁世界里，你无法知道是否还有别的线程正持有指向该节点的指针。因此释放时机成为一个独立的设计问题，ABA 只是它的一种表现形式。

[22] 视频画面时间区间：00:57:00–00:57:15。若为裁剪图，时间区间仍对应原视频画面。

7.4 本章小结

ABA 问题指值从 A 变回 A 而中间状态未被察觉，导致 CAS 误判成功。对策包括附加版本计数与推迟节点回收。内存回收时机是无锁设计的核心难点，ABA 只是其表现之一。

8 总结与延伸

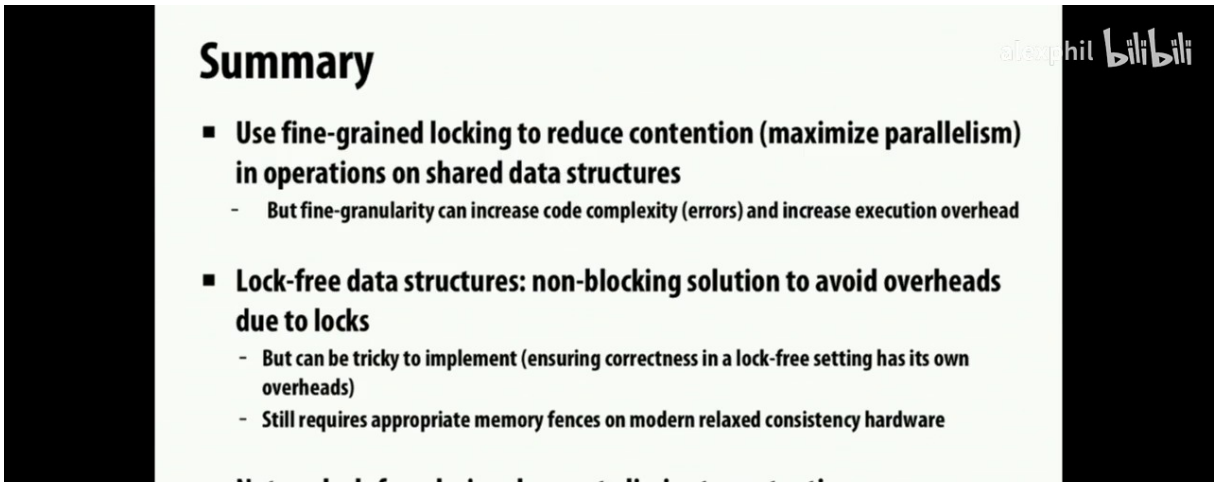


图 23: 总结：用细粒度锁降低争用，用无锁避免阻塞[23]

讲者在结尾给出参考资料，并说如果在维基百科上搜 ABA 问题或无锁队列，会找到相当不错的解释与文献链接。

把全讲串起来，这是一条从”锁太重”到”完全不用锁”的路径。pthread 互斥量开销可达数万周期，不适合应用级数据结构；自旋锁适合短暂持有，但把锁加在整个结构上会串行化所有访问。改进方向有三层：一把大锁、细粒度锁、完全无锁。比较并交换是三参数原子操作，返回旧值以判断成败，它既能实现无锁更新也能实现锁，是所有同步的硬件基础。C11 的 atomic 模板把原子性抽象成类型，但对复杂结构可能退化为加锁。并发链表的最简方案是一把锁，正确但完全串行；手递手细粒度锁的核心不变式是始终持有前一与当前两把锁，因此路径不会被抽走，顶层锁只用于让进入者有序，实用变体是用哈希桶把争用分散到短链表。无锁算法保证至少有一个线程总能推进，不受操作系统上下文切换影响，这个保证是系统层面而非个体层面。两个实例是单读单写的环形队列与链表队列，后者只有写者能分配释放节点。ABA 问题指值从 A 变回 A 而中间态未被察觉，使 CAS 误判成功；对策是版本计数与推迟回收，而内存回收时机是无锁设计的核心难点。

本讲义在讲者材料之上做的压缩，有四点需要与讲者原话区分：

- 无锁不等于无同步。它仍然依赖原子原语与内存序约束，只是不再依赖互斥。因此”去掉锁就更快”是误解：原子操作在高争用下同样昂贵，只是失败方式不同。
- 无锁的保证是系统级的，不保证个体线程的响应时间。若应用要求每个线程都在有限时间内推进，需要更强的无等待（wait-free）保证，而这类算法通常更难实现。
- 细粒度锁与无锁解决的是不同问题。前者降低争用范围，仍可能阻塞；后者消除阻塞，但引入 ABA 与回收难题。选择取决于应用更怕争用还是更怕阻塞。

[23] 视频画面时间区间：01:21:00–01:21:15。若为裁剪图，时间区间仍对应原视频画面。

- 节点回收是无锁设计的独立难题，不是 ABA 的附属。风险指针、引用计数、单写者负责释放都是为此存在。忽视这一点会让看似正确的无锁结构在长时间运行后崩溃。

这堂课的边界在于它是导论：讲者明确说这部分材料很棘手，建议回去反复核对。因此本文给出的是方案框架与问题清单，不含可直接照搬的完整实现。

两个可以直接从本讲数据提出的下一步问题：其一，ABA 问题可以用版本计数缓解，但计数器会回绕，那么回绕周期应当如何与并发规模匹配才安全？其二，单读单写的无锁队列依赖”只有一个写者”这一强限制，若要多生产者，需要哪些额外机制，代价是什么？